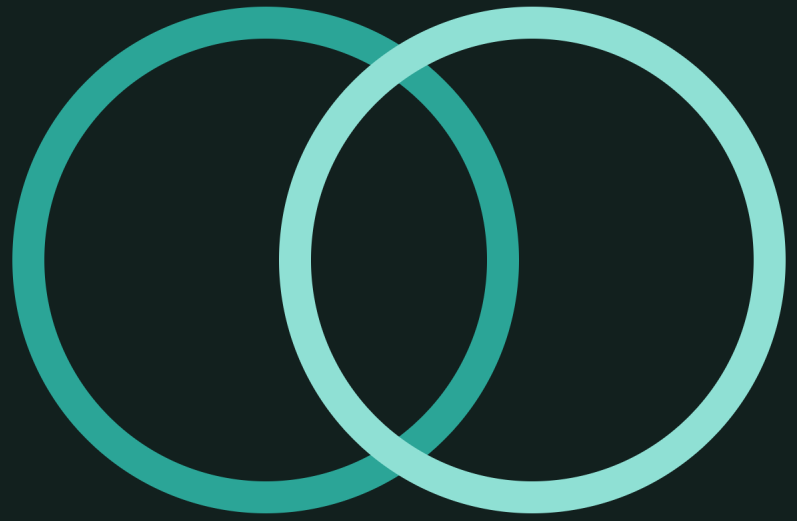
TRABAJO DE FIN DE MÁSTER · MÁSTER DE DESARROLLO CON IA

SharedExpenses

Gastos compartidos, cuentas claras y el mínimo de pagos para saldarlas.

Ayelén Stefanía Andrade · BIG School



EL PROBLEMA

Repartir gastos en grupo genera dudas

¿Quién pagó?

Los tickets se pierden entre chats, notas y fotos del móvil.

¿Cuánto me toca?

No todo se reparte igual: hay importes y porcentajes distintos.

¿Cómo saldarlo?

Con varias personas, cada uno acaba haciendo muchos pagos.

LA SOLUCIÓN

Registra, reparte y salda

Grupos

Registra por email y gestiona miembros y permisos.

Tres repartos

A partes iguales, por importes o por porcentajes.

Balances

Quién debe y a quién le deben; el grupo suma 0.

Liquidación

El mínimo de pagos para quedar en paz.

DEMO EN DIRECTO

Veámoslo en marcha

Aplicación: sharedexpenses-two.vercel.app

API y Swagger: sharedexpenses-dso8.onrender.com/swagger-ui.html

STACK TECNOLÓGICO

Tecnologías elegidas

Capa	Tecnología	Por qué
Backend	Java 21 · Spring Boot 3.5	APIs REST maduras y bien probadas
Datos	PostgreSQL · H2 · Flyway	SQL estándar y esquema versionado
Seguridad	Spring Security · JWT · BCrypt	JWT sin librerías externas
Frontend	React 19 · Vite · React Router	SPA ligera, fetch y CSS propio
Infraestructura	Docker · Render · Vercel	Despliegue gratuito y reproducible
Calidad	JUnit 5 · Mockito · MockMvc	Dominio, casos de uso y API

ARQUITECTURA

Monolito modular, capas hexagonales

Capa	users	groups	expenses	balances
infrastructure	Controllers y JPA	Controllers y JPA	Controllers y JPA	Controllers y JPA
application	Casos de uso	Casos de uso	Casos de uso	Casos de uso
domain	Java puro	Java puro	Java puro	Java puro

shared · seguridad (JWT, CORS, límite de peticiones, captcha) · errores ProblemDetail · configuración

PRINCIPIOS DE DISEÑO

SOLID, DRY, KISS y YAGNI en el código

S	Un caso de uso por clase: **CreateExpenseUseCase**, **LoginUseCase**…
O	Cada reparto es una **SplitStrategy**: un tipo nuevo es una clase nueva
L	Adaptadores JPA y dobles de test cumplen el mismo contrato del puerto
I	Puertos pequeños: **MemberBalanceProvider**, **GroupPaymentsRemover**…
D	Dominio y aplicación dependen de interfaces, nunca de Spring ni JPA
DRY	**Guard**, **Amounts** y **PasswordPolicy** reúnen reglas repetidas
KISS	Un monolito: sin microservicios, colas ni buses de eventos
YAGNI	Solo lo que se usa: nada de capas “por si acaso”

REGLAS DE NEGOCIO

Cada céntimo cuadra

Ejemplo: **10,00 €** a partes iguales entre 3

3,34 Ana · +1 céntimo	**3,33** Bea	**3,33** Carlos

El céntimo sobrante va al primer participante por id: el resultado no depende del orden en que se envían.

- Importes en **BigDecimal**, nunca double
- Las partes suman el total exacto
- Los porcentajes deben sumar 100
- Quien paga y participa es miembro
- Los balances del grupo suman 0

ALGORITMO DE LIQUIDACIÓN

El mínimo de pagos: como mucho N–1

Balances	
Ana	–151,34 € debe
Bea	+205,67 € le deben
Carlos	–54,33 € debe



Liquidación sugerida

Ana → Bea · **151,34 €**

Carlos → Bea · **54,33 €**

2 pagos para 3 personas

1. Se toman el mayor acreedor y el mayor deudor
2. El deudor paga lo máximo posible entre los dos
3. Uno de los dos queda a 0: se repite hasta terminar

SEGURIDAD

Defensa en capas

JWT y BCrypt

Sesión sin estado firmada con HS256; contraseñas con BCrypt.

Solo miembros

GroupAccess comprueba la pertenencia en cada caso de uso.

CORS

Solo el dominio del frontend puede llamar a la API.

Límite de peticiones

5 registros por hora e IP, 50 en total; 10 logins cada 5 minutos.

Captcha

Cloudflare Turnstile en el registro, verificado en el servidor.

RGPD

Borrar la cuenta anonimiza los datos sin romper el historial.

CALIDAD

Probado y versionado

150
tests automatizados

3
migraciones Flyway, sin perder datos

0
imports de Spring o JPA en el dominio

- **Dominio**, sin Spring: repartos, redondeo, balances y N−1
- **Casos de uso** con Mockito sobre los puertos
- **API** con MockMvc: 401, 403 a grupos ajenos, 429 y captcha
- **Persistencia** con @DataJpaTest y bloqueo optimista

DESPLIEGUE

Del repositorio a producción, gratis

GitHub
Cada push a main despliega automáticamente

→

Render
Imagen Docker multi-stage, perfil prod

→

Neon
PostgreSQL; Flyway migra al arrancar

Vercel
Build de Vite y rewrite para React Router

→

Navegador → API
La API está detrás de Cloudflare; CORS solo admite el dominio de Vercel

Plan gratuito de Render: el backend se duerme sin uso y la primera petición puede tardar hasta un minuto.

DESARROLLO ASISTIDO POR IA

Cómo he trabajado con IA

1

Yo defino

Requisitos, arquitectura y reglas de negocio, escritos en CLAUDE.md.

2

La IA implementa

Claude Code escribe código y tests en pasos pequeños.

3

Yo reviso

Pruebo cada paso, corrijo el rumbo y tomo las decisiones.

CLAUDE.md está en el repositorio: documenta las reglas que la IA tenía que seguir en cada paso.

RETOS Y APRENDIZAJES

Lo que aprendí por el camino

Reto	Solución
Tras Cloudflare, cada petición traía otra IP	Leer la IP real de CF-Connecting-IP
Un pago no corresponde a un gasto concreto	Historial de pagos y reglas claras al borrar
Definir reglas de negocio antes de implementar	Crear un archivo con reglas para entregar a la IA
Un esquema que cambia con el producto	Migraciones Flyway V2 y V3 sin perder datos

Gracias

Próximos pasos

- Transferir la propiedad de un grupo
- Detectar ediciones simultáneas de un gasto
- Notificaciones a los miembros

Enlaces

App: sharedexpenses-two.vercel.app

Código: github.com/stephieandrade/sharedexpenses

Vídeo: enlace en el README del repositorio